Block 1 · Tools and first project

Interface Development (0488) · 2nd DAM · 2026/2027

In this block

In UT 1 you learned the Kotlin you need. Now you start building user interfaces for Android with **Jetpack Compose**. This first block gives you the map: which tools and libraries are involved, how a Compose project is organised, how to read the code that Android Studio generates for you, and how to see your interface while you write it.

By the end of this block you will be able to:

- explain the difference between Views (XML) and Compose, and name the libraries and tools involved;
- open, run and explore a Compose project in Android Studio;
- read the code generated by the project template and explain what each part does;
- check your interface with `@Preview` without running the app;
- build a simple composable styled with modifiers;
- modify generated code safely, one small step at a time.

What you need: Android Studio, an emulator or a physical Android device, your fork of the course repository with the `UT2_Compose` project, and the Kotlin basics from UT 1.

Contenido

In this block 1
1.1 What is Jetpack Compose? 3
Imperative and declarative 3
1.2 Tools and libraries for building Android interfaces 3
1.2.1 Views (XML) and Compose 3
1.2.2 The libraries behind Compose 4
1.2.3 Tools in Android Studio 5
1.2.4 Where to learn more 5
1.3 Creating and opening the first project 5
Part A – Open the course project 5
Part B – How a project is created 6
Project structure 6
1.4 Reading the generated code 7
How to analyse generated code 8
1.5 Basic structure of a Compose app 9

Activity and Screen in separate files 9
1.6 Using the Preview 10
Previews and parameters 11
Running a preview on a device 11
1.7 Composables and modifiers 12
Text 12
Modifiers 12
The order of modifiers matters 13
The modifier parameter 13
1.8 First modifications to generated code 13
1.9 Practice 1 – My first Compose screen 14
Tasks 14
What you must deliver 15
Self-check questions 15
Key terms 15

1.1 What is Jetpack Compose?

Jetpack Compose is Android's modern toolkit for building native user interfaces. Instead of describing the screen in XML files and then changing it from code, you describe it with **Kotlin functions**. Compose draws what you describe and updates it when your data changes.

Imperative and declarative

With Views, the interface is **imperative**: you find a view and tell it what to change. With Compose, the interface is **declarative**: you describe what the screen must look like for the current data, and Compose takes care of the changes.

Views (imperative)	Compose (declarative)
The layout is in an XML file; the code finds the view and changes it.	The UI is a Kotlin function that receives data and describes the screen.

Views

```
// Views: XML + code that mutates the view
// <TextView android:id="@+id/greeting" ... />
findViewById<TextView>(R.id.greeting).text = "Hello $name!"
```

Compose

```
// Compose: the function describes the UI for the data it receives
@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name!")
}
```

> **Key idea**
> The UI is the result of applying a function to your data: **UI = f(state)**. When the data changes, Compose runs the function again and the screen follows. You will study this in depth in Block 3.

> **Connection with UT 1**
> Compose uses from the first line what you practised in UT 1: functions with **default and named arguments**, **trailing lambdas**, null safety and data classes. If any of them feels rusty, review it now.

1.2 Tools and libraries for building Android interfaces

1.2.1 Views (XML) and Compose

Both toolkits build native Android interfaces and can live in the same app. Google recommends Compose for new interfaces, and it is the toolkit we use in this module.

	Views (XML)	Jetpack Compose
Where the UI is defined	XML layout files, plus Kotlin or Java for the logic	Kotlin functions only
How the UI is updated	Imperative: find the view and change it	Declarative: describe the UI from the data; Compose updates it
Reusing pieces of UI	Custom view classes, `include`, fragments	Composable functions
Seeing the result while you work	Layout Editor with a palette of components	Preview next to the code; Layout Inspector on a running app

In Compose you do not drag components from a palette. You write the interface in code and Android Studio shows the result in the **Preview**.

1.2.2 The libraries behind Compose

Compose is not a single library but a set of them. You will see these names in the project's Gradle files (`app/build.gradle.kts` and `gradle/libs.versions.toml`):

Library	What it gives you

Compose UI
`androidx.compose.ui`
`androidx.compose.ui:ui-graphics`
	The basic building blocks: modifiers, drawing and graphics. Foundation (layouts, gestures) comes included with Material 3.

Material 3 `androidx.compose.material3`	Ready-made components that follow Material Design 3 (Text, Button, Card, Scaffold...) and the theme system (`MaterialTheme`).
Activity Compose `androidx.activity:activity-compose`	Provides `setContent`, which hosts composables inside an Activity.

Tooling
`androidx.compose.ui:ui-tooling-preview`
`androidx.compose.ui:ui-tooling`
	The `@Preview` annotation and the tools that show your composables in Android Studio.

Compose BOM `androidx.compose:compose-bom`	A "bill of materials": one version number that keeps all Compose libraries compatible with each other.

Compose is also switched on in the module's `app/build.gradle.kts` file, inside the `android` block. If this option is missing, the project does not compile composables:

```
android {
    buildFeatures {
        compose = true
    }
}
```

The version numbers are not written in `app/build.gradle.kts`: they live in `gradle/libs.versions.toml`, the **version catalog**. Open it in your project and look at the versions of Android Gradle Plugin (`agp`), Kotlin and `composeBom`. They change with every Android Studio release, so never copy versions from a tutorial: use those of your own project.

1.2.3 Tools in Android Studio

Tool	What it is for	You will use it
Preview	Shows your composables next to the code, without running the app. Modes: Code, Split and Design.	From this block
Device Manager	Creates and starts virtual devices (emulators).	From this block
Physical device	Runs the app on a real phone with USB debugging enabled.	Optional
Layout Inspector	Inspects a running app: the tree of components and the properties of each one.	From Block 2

1.2.4 Where to learn more

- The official documentation: **developer.android.com/compose**.
- The official Google codelabs "Getting started with Compose: building a basic UI" and "Jetpack Compose basics". Complete them before moving on to the next block: they give you a good overview of the new environment.

Looking ahead. With **Kotlin Multiplatform (KMP)**, Compose can also be used to build interfaces for other platforms, not only Android. What you learn in this module is the base for that.

1.3 Creating and opening the first project

Part A – Open the course project

1. Open Android Studio and choose **File > Open**.

2. Select the `UT2_Compose` folder of your fork (the one that contains `settings.gradle.kts`). Do not select the root folder of the repository.
3. Wait for the first **Gradle sync** to finish. It downloads many dependencies, takes several minutes and needs an internet connection. Do not interrupt it.
4. Choose a device: an emulator created in **Device Manager**, or your phone connected by USB.
5. Press **Run** and check that the app starts.

Part B – How a project is created

Android Studio creates a new project from a template. For Compose we use the **Empty Activity** template, which already includes Compose and Material 3. The wizard asks for:

Field	Meaning
Name	The name of the app.
Package name	A unique identifier for the app, written like a reversed domain name.
Save location	The folder where the project is created.
Minimum SDK	The oldest Android version the app supports.
Build configuration language	Kotlin DSL (`.kts` files), the one we use.

The screens of the wizard change slightly between Android Studio versions, but the fields are the same.

Project structure

```
UT2_Compose/
|-- settings.gradle.kts
|-- build.gradle.kts
|-- gradle/
|   \-- libs.versions.toml
\-- app/
    |-- build.gradle.kts
    \-- src/main/
        |-- AndroidManifest.xml
        |-- java/com/example/myfirstcomposeapp/
        |   |-- MainActivity.kt
        |   \-- ui/theme/   (Color.kt, Theme.kt, Type.kt)
        \-- res/
```

File or folder	What it contains
`settings.gradle.kts`	Which modules the project has (here, `app`).
`gradle/libs.versions.toml`	The version catalog: the versions of the libraries in one place (present in all current templates).
`app/build.gradle.kts`	The configuration of the app module and its dependencies.
`AndroidManifest.xml`	Declares the app and its screens (Activities).

File or folder	What it contains
`MainActivity.kt`	The entry point of the app. The `java` folder also holds Kotlin code.
`ui/theme/`	The colours, typography and theme of the app.
`res/`	Resources: images, strings, icons.

1.4 Reading the generated code

The template generates working code. Before changing anything, you must be able to explain what it does. This is the code generated for the course project (imports omitted). If you create another project with a newer Android Studio, it may differ a little, but the structure is the same.

MainActivity.kt

```
class MainActivity : ComponentActivity() {                          // (1)
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()                                          // (2)
        setContent {                                                // (3)
            MyFirstComposeAppTheme {                                    // (4)
                // (5) Scaffold and innerPadding
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable                                                         // (6)
fun Greeting(name: String, modifier: Modifier = Modifier) {         // (7)
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true)                                     // (8)
@Composable
fun GreetingPreview() {
    MyFirstComposeAppTheme {
        Greeting("Android")
    }
```

```
}
```

	Explanation
(1)	The app's screen is an **Activity**. `ComponentActivity` is the base class that supports Compose.
(2)	Lets the app draw behind the system bars (status bar and navigation bar). Recent templates include it; older ones do not.
(3)	`setContent` is the bridge between the Android world and Compose. Everything inside the braces is composable code.
(4)	The **theme** of the project (its name comes from the name of your app). It applies colours, typography and shapes to everything inside. It is defined in `ui/theme/Theme.kt`.
(5)	`Scaffold` gives the basic structure of a screen. It passes `innerPadding` so the content does not hide behind the system bars. You will study it in Block 7.
(6)	`@Composable` marks a function that describes part of the UI.
(7)	`Greeting` receives the data it needs (`name`) and an optional `modifier` with a default value. The `modifier` parameter is a convention you will see in every composable.
(8)	`@Preview` tells Android Studio to show this function in the Preview. `GreetingPreview` is only for the IDE: the app never calls it.

How to analyse generated code

1. **What does it do?** Find the entry point and follow what is called from there.
2. **Who wrote each part?** Separate your code from the library code you import.
3. **What depends on what?** Which functions receive data, and where does that data come from?
4. **What can I change safely?** Change one thing at a time and check the result.

> **Generated code is a starting point**
> Templates change between versions, and the same is true of code you find in tutorials or get from an AI assistant. Never use code you cannot explain: read it, run it and test it before you build on it.

> **Exercise 1.2**
>
> Answer these questions about the generated code. You will include the answers in your Practice 1 report.
>
> 1. Which class is the entry point of the app?
> 2. What does `setContent` do?
> 3. Where is the theme defined? What changes if you remove the `MyFirstComposeAppTheme { }` wrapper? Try it and look at the result.
> 4. Why does `Greeting` have a `modifier` parameter?
> 5. What is the difference between `Greeting` and `GreetingPreview`? Which one runs on the device?
> 6. Look at the `import` lines: which libraries does this code use?

1.5 Basic structure of a Compose app

A Compose interface is a **tree of composable functions** that call one another. In the template, the tree looks like this:

```
setContent
\-- MyFirstComposeAppTheme
    \-- Scaffold
        \-- Greeting
            \-- Text
```

Composable functions follow a few rules:

- They are marked with `@Composable`.
- They can only be called from other composable functions, or from `setContent`.
- They **emit** UI instead of returning it. Their return type is normally `Unit`.
- Their name is a noun in PascalCase, such as `Greeting` or `ProfileCard`.
- The data they need arrives as **parameters**. With the same parameters, they describe the same UI.
- Keep them small: split the screen into small composables that you can reuse and preview separately.

Activity and Screen in separate files

In the template, `Greeting` lives in the same file as `MainActivity`. In real projects each screen goes in its own file, usually called `<Name>Screen.kt`. The Activity only starts the app and hosts the screen; the file of the screen holds the composables that describe it. This keeps files short and easy to read. For a login screen:

```
// MainActivity.kt: starts the app and shows the screen
setContent {
    MyFirstComposeAppTheme {
        LoginScreen()
    }
}

// LoginScreen.kt: the composables that describe the screen
@Composable
fun LoginScreen(modifier: Modifier = Modifier) {
    // ...
}
```

In Practice 1 everything goes in one file to keep it simple; from the next blocks you will build one screen per file.

> **Coming in Block 3**
> When the data that a composable reads changes, Compose calls that function again. This is called **recomposition**. For now, remember that composables can run more than once, so they should only describe UI.

1.6 Using the Preview

The Preview shows your composables inside Android Studio, without installing the app on a device. You can switch between **Code**, **Split** and **Design** with the buttons at the top right of the editor. If the Preview does not update, use **Build & Refresh**.

A preview is a composable function with no parameters, annotated with `@Preview`. It should wrap its content in the theme of the project.

`@Preview` parameter	What it does
`name`	A label shown above the preview. Useful when you have several.
`showBackground`	Paints a background behind the composable.
`widthDp`, `heightDp`	Sets the size of the preview.
`uiMode`	Light or dark mode, for example `Configuration.UI_MODE_NIGHT_YES`.
`showSystemUi`	Shows the status bar and the navigation bar.
`fontScale`	Simulates a larger or smaller system font.
`locale`	Simulates another language, for example `"es"`.

You can add several `@Preview` annotations to the same function. This example shows the same composable in light and dark mode:

Two previews of the same composable (needs import android.content.res.Configuration)

```
@Preview(name = "Light")
@Preview(name = "Dark", uiMode = Configuration.UI_MODE_NIGHT_YES)
@Composable
fun GreetingPreviews() {
    MyFirstComposeAppTheme {
        Surface {
            Greeting("Android")
        }
    }
}
```

`Surface` paints a background and sets the text colour from the theme, so the dark preview looks right. You will meet it again in Block 4.

Previews and parameters

A `@Preview` function cannot receive parameters. If the composable that you want to see needs data, create a small preview function that calls it with example values:

```
@Composable
fun ProfileCard(name: String, age: Int) {
    // ...
}

@Preview(showBackground = true)
@Composable
fun ProfileCardPreview() {
    MyFirstComposeAppTheme {
        ProfileCard(name = "Ana", age = 20)
    }
}
```

For previews with many sets of data there is the `@PreviewParameter` annotation, which you can look up in the official documentation when you need it.

Running a preview on a device

You can also run a single composable on the emulator or the device, without starting the whole app or going through the screens that lead to it. Use the option to run a preview on a device, which Android Studio offers in the Preview panel and next to the `@Preview` annotation. It is useful for checking a screen that is far from the start of the app.

Some Android Studio versions also offer **Live Edit**, which updates the app running on the device while you edit the code. It is enabled in the settings of Android Studio, and its behaviour changes between versions, so check it in yours.

The Preview also has an **Interactive mode**, which lets you click inside the preview. The Preview does not replace running the app: use it to work fast, and run the app on the emulator or the device to confirm.

1.7 Composables and modifiers

Text

`Text` is the basic component to show text. Some of its parameters:

```
Text(
    text = "Hello Compose",
    color = Color.DarkGray,
    fontSize = 24.sp,
    fontWeight = FontWeight.Bold,
    textAlign = TextAlign.Center
)
```

Sizes use **dp** (density-independent pixels), for example `16.dp`. Text sizes use **sp** (scalable pixels), for example `24.sp`, which also respect the font size chosen by the user. Remember to import `androidx.compose.ui.unit.dp` and `sp`.

Modifiers

A `Modifier` changes how a composable looks or behaves: its size, padding, background, border or click. You chain modifiers with dots:

```
Text(
    text = "Hello",
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
)
```

If you know XML: there are no `match_parent` or `wrap_content` attributes in Compose. `fillMaxWidth()`, `fillMaxHeight()` and `fillMaxSize()` do what `match_parent` did, and a composable takes the size of its content by default, as `wrap_content` did.

Modifier	What it does
`padding(16.dp)`	Adds space around the content.
`size(100.dp)`, `width()`, `height()`	Sets a fixed size.
`fillMaxWidth()`, `fillMaxSize()`	Fills the available width, or all the available space.
`background(Color.Yellow)`	Paints a background colour.
`border(2.dp, Color.Black)`	Draws a border.

Modifier	What it does
`clip(RoundedCornerShape(8.dp))`	Cuts the content to a shape, for example with rounded corners.
`clickable { ... }`	Makes the composable respond to clicks.

The order of modifiers matters

Modifiers are applied one after another, from the outside in. Compare these two texts:

```
// 1. The background covers the text AND the padding
Text("Hello", modifier = Modifier.background(Color.Yellow).padding(16.dp))

// 2. The padding is outside; the background covers only the text
Text("Hello", modifier = Modifier.padding(16.dp).background(Color.Yellow))
```

In the first one the yellow area includes the space; in the second one the space stays outside the yellow area. Try both in the Preview and see the difference.

The modifier parameter

By convention, a composable that you write receives a `modifier: Modifier = Modifier` parameter and applies it to its root element. This lets the code that calls it change its size, padding or position.

```
@Composable
fun SectionTitle(text: String, modifier: Modifier = Modifier) {
    Text(
        text = text,
        fontSize = 24.sp,
        fontWeight = FontWeight.Bold,
        modifier = modifier.padding(16.dp)
    )
}
```

1.8 First modifications to generated code

Now you change the generated code yourself. Work in small steps. After each step, check the Preview and make a commit with a clear message.

1. **Change the text.** In `Greeting`, change `"Hello $name!"` and check that the Preview updates.
2. **Add a parameter with a default value.** Make the greeting word a parameter and call it with named arguments:

```
@Composable
fun Greeting(name: String, greeting: String = "Hello", modifier: Modifier = Modifier) {
    Text(text = "$greeting $name!", modifier = modifier)
}

// Calls:
Greeting(name = "Android")
Greeting(name = "Compose", greeting = "Welcome")
```

3. **Add a second preview** in dark mode, as shown in section 1.6.
4. **Apply modifiers.** Add padding and a background, and try the two orders from section 1.7.
5. **Run the app** on the emulator or the device and compare it with the Preview. Are they the same?

> **Working method**
> Change one thing at a time. Check the Preview. Commit. If something breaks, you know exactly which change caused it.

1.9 Practice 1 – My first Compose screen

Where: project `UT2_Compose` of your fork, package `com.example.myfirstcomposeapp.exercises.intro`, file `Practice1.kt`.

Tasks

Task	What to do
A	**Analyse.** Answer the questions of Exercise 1.2 in your report.
B	**Modify.** Write a version of `Greeting` with a `greeting` parameter that has a default value. Add two previews: light and dark.
C	**Create.** Write a composable `InfoTag(text, backgroundColor, modifier)` that shows bold white text on a coloured background, with padding and rounded corners. Add three previews: a short text, a long text and a different colour.
D	**Experiment.** Make two previews that show the two orders of `background` and `padding`. Take a screenshot of each and explain in two or three sentences what you see.
E	**Run.** Create a `Practice1Screen()` composable that shows an `InfoTag`, make `MainActivity` display it, and run it on the emulator or the device.

At this point you can only place one element on the screen. In Block 2 you will learn how to place several.

What you must deliver

- The code, committed to your fork of the course repository, with clear commit messages.
- A demonstration of the app running on the emulator or a device.
- A short report named `practice1-report.md`, next to your code, with your answers to Task A, the screenshots of Task D and the emulator or device, and the difficulties you found.
- If you used an AI assistant, the declaration of use (see below).

Your teacher may ask you to explain your code in person.

> **Using AI in your practices**
>
> You may use an AI assistant as a support tool in your practices, but you must **declare it** in your report: the tool you used, what you used it for, which parts of the code were affected, what you changed yourself, and how you checked that it works.
>
> You must be able to explain every line of what you hand in. AI is not allowed in written tests or in oral defences.

Self-check questions

1. What is the difference between an imperative and a declarative interface?
2. Name three libraries that a Compose project uses and say what each one is for.
3. What does `setContent` do?
4. What is the `@Composable` annotation for? Where can a composable function be called from?
5. Why does the order of `background` and `padding` change the result?
6. Is `GreetingPreview` part of the app that runs on the device? Why?

Key terms

English	Español	Section
Composable function	Función componible	1.1, 1.5
Declarative UI	Interfaz declarativa	1.1
Recomposition	Recomposición	1.5
Modifier	Modificador	1.7
Preview	Vista previa	1.6
Theme	Tema	1.4
Template	Plantilla	1.3

English	Español	Section
Emulator	Emulador	1.3
Gradle sync	Sincronización de Gradle	1.3
State	Estado	1.1